

Zapis do pliku JSON

1. Skąd aplikacja wie, gdzie ma zapisywać plik?

Początek tej mechaniki jest w `MauiProgram.cs`:

```
var filePath = Path.Combine(FileSystem.AppDataDirectory, "notes.json");
builder.Services.AddSingleton(new JsonStore(filePath));
```

Tu dzieją się dwie ważne rzeczy:

- tworzona jest ścieżka do pliku `notes.json`,
- ta ścieżka trafia do klasy `JsonStore`.

To znaczy, że `JsonStore` od początku wie, **do jakiego konkretnego pliku ma zapisywać dane**. Sama metoda zapisu nie zgaduje miejsca pliku. Ścieżka jest ustalana wcześniej i przekazywana do klasy odpowiedzialnej za dane.

2. `JsonStore` zapamiętuje ścieżkę pliku

W konstruktorze `JsonStore`:

```
public JsonStore(string filePath)
{
    _filePath = filePath;
    if (!File.Exists(_filePath))
    {
        File.WriteAllText(_filePath, "[]");
    }
}
```

Tutaj:

- parametr `filePath` trafia do pola `_filePath`,
- jeśli plik jeszcze nie istnieje, aplikacja tworzy go od razu z zawartością `[]`.

To oznacza, że program od początku przygotowuje sobie pusty plik JSON zawierający pustą listę notatek.

Zasada: najpierw przygotuj plik, potem dopiero zapisuj do niego dane.

3. Użytkownik uruchamia zapis

Start tej mechaniki jest w `MainPage.xaml`:

```
<Button x:Name="AddButton" Text="Dodaj" Clicked="Add_Clicked" />
```

Kliknięcie przycisku uruchamia metodę:

```
private async void Add_Clicked(object sender, EventArgs e)
```

Czyli przyczyna jest bardzo prosta:

użytkownik klika przycisk

→ **program wchodzi do `Add_Clicked`**

4. Add_Clicked pobiera dane z interfejsu

Na początku metody:

```
var text = TextEntry.Text?.Trim();
if (string.IsNullOrEmpty(text))
{
    await DisplayAlert("Info", "Napisz coś...", "OK");
    return;
}
```

Tutaj aplikacja:

- pobiera tekst z pola `Entry`,
- usuwa zbędne spacje,
- sprawdza, czy tekst nie jest pusty.

To ważne, bo zapis ma sens tylko wtedy, gdy istnieją sensowne dane do zapisania.

5. MainPage nie zapisuje pliku sama

To bardzo ważne.

W `Add_Clicked` nie masz bezpośrednio:

```
File.WriteAllTextAsync(...)
```

Zamiast tego masz:

```
await _store.SaveNoteAsync(new Note { Text = text });
```

albo przy edycji:

```
await _store.SaveNoteAsync(_editingNote);
```

To znaczy, że `MainPage` nie zajmuje się technicznym zapisem pliku.
Ona tylko przekazuje dane do klasy `JsonStore`.

Czyli masz rozdzielenie odpowiedzialności:

- `MainPage` obsługuje interfejs i decyzję użytkownika,
- `JsonStore` odpowiada za pracę na pliku.

To jest jedna z najważniejszych rzeczy.

6. SaveNoteAsync przygotowuje nowy stan danych

W `JsonStore.cs`:

```
public async Task SaveNoteAsync(Note note)
{
    var notes = await GetNotesAsync();
    if (note.Id == 0)
    {
        var nextId = notes.Count == 0 ? 1 : notes.Max(n => n.Id) + 1;
        note.Id = nextId;
        notes.Add(note);
    }
}
```

```

else
{
    var idx = notes.FindIndex(n => n.Id == note.Id);
    if (idx >= 0) notes[idx] = note;
    else notes.Add(note);
}
await WriteAllAsync(notes);
}

```

Ta metoda robi kilka ważnych kroków.

a) Najpierw odczytuje aktualną listę notatek

```
var notes = await GetNotesAsync();
```

To znaczy, że program nie zapisuje jednej notatki „w próżni”.
Najpierw pobiera cały obecny stan pliku.

Czyli działa tak:

- odczytaj obecną listę,
- zmień ją w pamięci,
- zapisz całość ponownie.

To typowy model pracy z plikami JSON w prostych aplikacjach.

b) Potem decyduje: dodać czy zaktualizować

Jeśli notatka ma `Id == 0`, to jest nowa:

```

var nextId = notes.Count == 0 ? 1 : notes.Max(n => n.Id) + 1;
note.Id = nextId;
notes.Add(note);

```

Jeśli notatka ma już `Id`, to jest edytowana i trzeba ją znaleźć na liście:

```

var idx = notes.FindIndex(n => n.Id == note.Id);
if (idx >= 0) notes[idx] = note;
else notes.Add(note);

```

Dla mechaniki zapisu kluczowe jest to, że po tych operacjach program ma już gotową, zaktualizowaną listę `notes`.

7. Właściwy zapis zaczyna się w `WriteAllAsync`

Na końcu `SaveNoteAsync`:

```
await WriteAllAsync(notes);
```

To jest moment przejścia z logiki aplikacji do właściwego zapisu pliku.

Można to powiedzieć tak:

`SaveNoteAsync` przygotowuje dane, a `WriteAllAsync` naprawdę zapisuje je do pliku.

8. WriteAllAsync zamienia obiekty na tekst JSON

Kod:

```
private async Task WriteAllAsync(List<Note> notes)
{
    var json = JsonSerializer.Serialize(notes, _jsonOptions);
    await File.WriteAllTextAsync(_filePath, json);
}
```

Tu są dwa główne kroki.

Krok 1 — serializacja

```
var json = JsonSerializer.Serialize(notes, _jsonOptions);
```

To jest bardzo ważna linia.

W pamięci programu `notes` jest listą obiektów C# typu `Note`.
Plik jednak nie przechowuje obiektów C#, tylko tekst.

Dlatego trzeba zamienić:

- `List<Note>`
na
- tekst JSON.

To właśnie robi `JsonSerializer.Serialize(...)`.

Przykładowo lista obiektów może zamienić się na coś takiego:

```
[
  {
    "Id": 1,
    "Text": "Pierwsza notatka"
  },
  {
    "Id": 2,
    "Text": "Druga notatka"
  }
]
```

Czyli **serializacja** to zamiana danych programu na format tekstowy nadający się do zapisania w pliku.

Krok 2 — zapis tekstu do pliku

```
await File.WriteAllTextAsync(_filePath, json);
```

Tu następuje rzeczywisty zapis.

Ta linia:

- bierze ścieżkę `_filePath`,
- bierze gotowy tekst JSON,
- zapisuje go do pliku.

To właśnie ta linia naprawdę zapisuje dane na dysku.

Wszystko wcześniej to było:

- zebranie danych,
- przygotowanie listy,
- zamiana na JSON.

Dopiero tu tekst trafia do pliku.

Co dokładnie zapisuje się do pliku?

W głównym pliku `notes.json` zapisywana jest lista obiektów `Note`.

Klasa `Note` wygląda tak:

```
public class Note
{
    public int Id { get; set; }
    public string Text { get; set; } = "";
}
```

To oznacza, że każda notatka w JSON będzie miała:

- `Id`,
- `Text`.

Czyli struktura pliku zależy bezpośrednio od modelu `Note`.

Model danych wpływa na to, jak wygląda JSON.

Odświeżenie widoku po zapisie

Po zapisaniu notatki w `Add_Clicked` masz:

```
TextEntry.Text = "";
await RefreshAsync();
```

A w `RefreshAsync`:

```
private async Task RefreshAsync()
{
    NotesView.ItemsSource = await _store.GetNotesAsync();
}
```

To nie jest już sam zapis, ale to ważny etap mechaniki końcowej:

- dane zostały zapisane,
- lista jest ponownie odczytywana z pliku,
- `CollectionView` dostaje nowy stan.

Czyli użytkownik od razu widzi efekt zapisu.

To można opisać tak:

zapis do pliku

→ **ponowny odczyt danych**

→ **odświeżenie listy na ekranie**